Orchestrating MCP-Based Scheduling Assistants: Comparing LLM and Backend Control

Daniel Becker[1,2]

[1]Coburg University of Applied Sciences and Arts, Germany
[2]HUK-COBURG, Germany

Abstract

Large language models can use software tools, but successful tool calls do not necessarily produce a correct task outcome. This paper compares three orchestration variants for an MCP-based conversational scheduling assistant: individual tools with a planner, bundled backend tools with a planner, and individual tools without a separate planner. The configuration-selection study evaluates three models on two scenarios, separating technical execution, correct tool use and outcome correctness while also measuring response time and model-call cost. In the tested scenarios, Claude Haiku 4.5 without a separate planner produced correct outcomes in all regular runs. Backend orchestration reduced time and cost for this model but produced fewer correct outcomes. The main contribution is a case study showing why orchestration choices must be assessed through complete task outcomes alongside execution efficiency. Shared application-side validation and explicit user confirmation constrain all variants. The small, task-specific comparison supports prototype selection, not an independent model ranking or proof of production readiness.

1. Introduction

Large language models (LLMs) can use external tools to retrieve information and invoke software functions [SDYD*23]. In a conversational scheduling assistant, this capability must support a sequence of dependent actions: interpreting preferences, clarifying missing information, retrieving suitable options and confirming a booking. A plausible response alone is insufficient. The application must also retain dialogue context, validate inputs and keep the final booking decision under human control.

Research on tool-using LLMs addresses both the use of individual APIs and the coordination of several steps. ToolLLM studies model training and evaluation for single-tool and multi-tool tasks [QLY*24]. ReAct combines reasoning with actions so that observations can inform later decisions [YZY*23]. RestGPT separates planning, API selection and execution to handle requests involving RESTful APIs [SXZ*23]. These approaches motivate studying how tool calls are coordinated, alongside whether they execute successfully. For a scheduling application, the comparison must also consider whether the complete process produces the intended outcome and how much time and cost it requires.

The Model Context Protocol (MCP) provides a common interface through which applications can access tools exposed by servers [Mod25]. For the scheduling task considered here, a further design decision concerns orchestration: which component selects and sequences the required operations? The LLM can coordinate individual tools, or a backend tool can bundle several operations into a predefined sequence. A separate planning stage introduces another design choice whose value must be assessed alongside its additional processing effort.

This paper investigates the following question: *How do three MCP-based orchestration variants compare in outcome correctness, response time and model-call cost in a conversational scheduling workflow?* We examine a prototype that lets users access existing services through a chat interface. It stores relevant information from the conversation in a structured form. Users must explicitly confirm an appointment before it is booked. The selection stage compares LLM orchestration with a planner, backend orchestration with a planner, and LLM orchestration without a separate planner (Figure 2). Each variant is tested with three models on two scheduling scenarios as part of configuration selection.

The main contribution is an empirical comparison of these three orchestration variants in a clearly defined scheduling workflow. It distinguishes technical execution, correct tool use and outcome correctness, and considers response time and model-call cost alongside these measures. The central finding is that technically successful tool execution does not necessarily produce a correct scheduling outcome. The shared architecture provides the basis for this comparison; the later settings, functional, robustness, user and load assessments provide supporting evidence about the selected prototype. All variants use MCP, so the study evaluates orchestration choices rather than whether the protocol itself provides an advantage. The following sections present the background and requirements, system design, evaluation, and implications for comparable applications.

2. Background and Related Work

MCP separates the host application's coordination responsibilities from the capabilities exposed by servers [Mod25]. In this paper, dialogue state denotes the structured information retained across turns. Four studies inform the use of tools, their orchestration and user control.

ToolLLM. The framework addresses the training and evaluation of language models for real-world API use. The ToolBench dataset covers 16,464 RESTful APIs, while a depth-first search method explores alternative sequences of calls. The resulting ToolLLaMA model demonstrates multi-tool execution and generalisation to unseen APIs in the reported evaluation [QLY*24]. This supports studying tool selection and sequencing together. Our prototype instead uses existing models with a small, predefined set of MCP tools and requires no model training.

RestGPT. This approach separates task planning, API selection and execution. The system uses API descriptions to construct calls and process responses, then feeds execution results back into planning. On RestBench, covering TMDB and Spotify tasks, removing the planner reduces success rates [SXZ*23]. This motivates our comparison of orchestration with and without a separate planner. The reported benefit applies to the evaluated benchmark and architecture, so its transfer to a bounded scheduling workflow requires evaluation.

AI Chains. This approach connects multiple model calls so that each step can use earlier outputs. An interactive interface exposes intermediate results and allows users to modify individual steps. A study with 20 participants found improved task outcomes and perceived transparency, controllability and collaboration compared with a non-chaining interface [WTC22]. For our design, this supports storing intermediate state and allowing corrections. How to divide tool orchestration between the LLM and backend remains an open question.

Guidelines for Human-AI Interaction. This work presents 18 design guidelines evaluated through several rounds, including a study with 49 practitioners examining 20 AI-enabled products. The guidelines address communicating capabilities, correcting errors, clarifying uncertain goals and remembering recent interactions [AWV*19]. These principles inform our dialogue design. The requirement for explicit confirmation before booking is our application-specific implementation of human control.

Together, these studies connect tool capability, modular processing and human oversight. Our study applies these perspectives to an MCP-based scheduling assistant and compares three implementations that distribute planning and execution between the LLM and backend. Its focus is the resulting reliability and efficiency trade-offs within this controlled workflow.

3. Use Case and Requirements

An authorised staff member uses the assistant to arrange an appointment during a customer interaction. The customer is already identified and interested in an appointment. The staff member enters time and location preferences, clarifies missing information and reviews available options with the customer. Preferences can be revised and the process cancelled before booking. Selecting a returned option confirms booking after the customer's agreement; no additional confirmation turn is required. The application validates the request and reports success only after service confirmation. A confirmation email can follow if a valid address is available. Direct customer access, professional advice and changes to confirmed bookings are outside scope.

Dialogue information must persist across turns and remain correctable; changes must invalidate dependent results. Access is restricted to authorised users and permitted records, and model input is limited to task-relevant information. Tool parameters and booking conditions require application-side checks. Empty search results must be distinguished from technical failures. Responses and delays should support an ongoing customer conversation.

4. System Design and Orchestration Variants

This section describes the shared architecture and dialogue state before introducing the three orchestration variants.

4.1. Architecture and Dialogue State

The prototype adds a conversational interface to existing services, as shown in Figure 1. The dialogue client acts as the MCP host, manages the interaction and invokes the language model. An embedded MCP client communicates with a separate MCP server, which exposes the scheduling functions as tools and accesses existing services through their APIs. The application executes model-

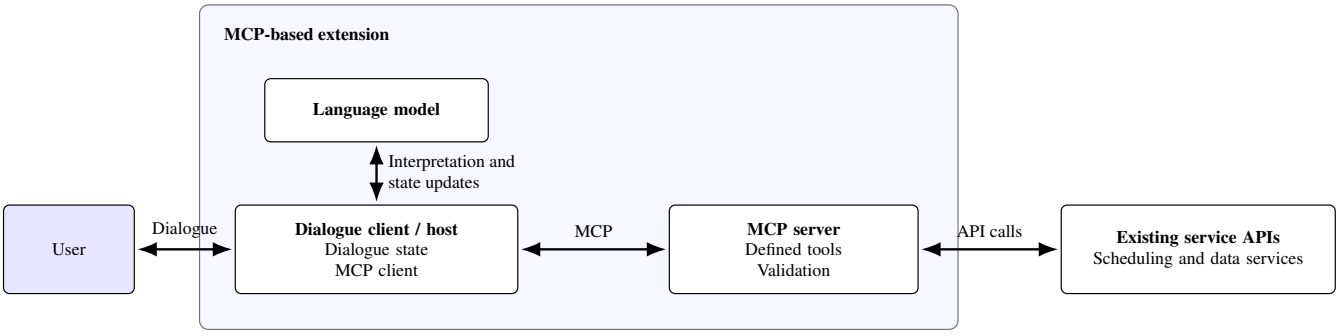


Figure 1: Shared MCP architecture. The dialogue client manages state and model calls; the server validates tool requests and accesses existing services. Arrows show requests and responses. The model has no direct service access. Internal deployment details are omitted.

selected tool calls through this connection. The language model has no direct access to the underlying services.

The dialogue client maintains structured state containing known preferences, missing information and the current processing step. Each new input is interpreted together with this state, producing a response and an updated state for the next turn. The server separately retains intermediate service results needed by subsequent operations. This separation allows reusable results to remain available without placing all service data in the model context. Corrections update the relevant state and invalidate dependent results before processing continues.

4.2. Orchestration Variants

The three variants in Figure 2 share this architecture but differ in tool granularity and the use of a separate planning stage.

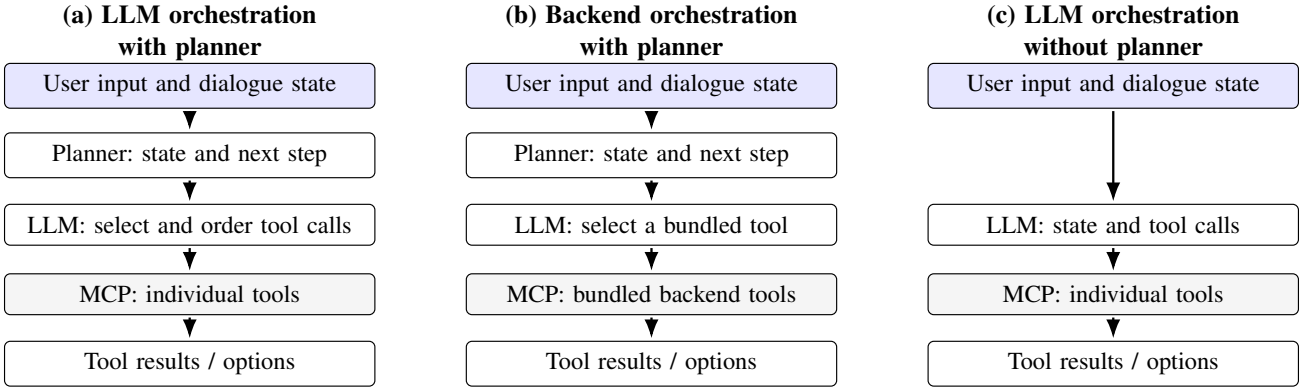


Figure 2: Three orchestration variants: **(a)** individual tools with a planner, **(b)** bundled backend tools with a planner, and **(c)** individual tools without a separate planner. Blue marks inputs; grey marks tools. Arrows show processing for complete inputs. All variants retain dialogue state, clarification, explicit booking confirmation and application-side validation. Clarification and confirmation paths are omitted.

LLM orchestration with a planner. Each new user input first triggers a model call that updates dialogue state and identifies the next task step. This planner does not select or execute tools. If information is missing, the application requests clarification. Otherwise, a subsequent model interaction selects and sequences individual MCP tools using their descriptions and the task instructions. Several dependent tool calls may follow, with the planner invoked again only when a new user input arrives.

Backend orchestration with a planner. The same planning stage precedes tool selection, but the exposed tools bundle several operations. The language model selects a bundled tool, while the backend determines the sequence of its internal steps and passes intermediate results between them. This reduces the number of individual operations that the model must coordinate. Tool selection remains model-driven, so backend orchestration does not make the entire workflow deterministic.

LLM orchestration without a planner. Individual MCP tools remain available, but the separate planning call is omitted. State updates, clarification and tool selection are handled together during model-driven dialogue processing. Tool results inform subsequent operations and the response to the staff member.

All variants retain application-side validation and the confirmation described in Section 3. The server resolves the selected option against stored search results before booking. Tool selection by the model therefore remains distinct from permission to execute the operation.

5. Evaluation Methodology

The study separates configuration selection from evaluation of the selected prototype; additional development checks are excluded from standardised run counts.

5.1. Configuration Selection

Three tool-capable models were tested with each variant: Claude Haiku 4.5 and Qwen3-235B-A22B-2507 at temperature 0, and Amazon Nova Pro at 0.00001. Additional reasoning was disabled for all three.

Two standardised scenarios cover different information requirements. In T01, a complete request is followed by selection and confirmation of a proposed option, giving two user inputs. In T02, an incomplete request requires clarification, followed by the missing information and then confirmation, giving three inputs.

Each scenario–model–variant combination was executed six times, giving 108 runs. The first run followed an application restart and was treated separately as a cold start. The remaining five repetitions per combination yielded 90 regular runs for analysis. Backend orchestration without a planner was not tested.

5.2. Selection Criteria and Measures

Technical success, tool behaviour and result correctness were assessed separately. Technical success requires the intended tool calls to complete successfully. Correct tool behaviour requires the necessary calls without missing, erroneous or unnecessarily repeated invocations. Result correctness requires a valid scheduling outcome

and the confirmation email specified in the test scenario. Additional read-only calls do not automatically constitute an incorrect outcome, whereas repeated state-changing calls can affect correctness. These measures were evaluated across all five regular repetitions.

For each input, response time was measured from submission until the full response was displayed. These times were added across the scenario, excluding the time users spent reading and entering information. Token usage and model-call cost were accumulated across all model calls, accounting for input, output and applicable cache-token categories. Reported response times are medians of technically successful regular runs, including those with incorrect outcomes. Token and cost medians include all five regular runs, since failed attempts also consume resources. In Table 1, relative cost is the ratio of median model-call costs, normalised to LLM orchestration with a planner within each model and scenario. The small number of repetitions allows a descriptive comparison, but does not support broad statistical conclusions.

5.3. Evaluation of the Selected Configuration

Selection prioritised correctness and reliability, followed by response time and cost. Five Haiku settings used individual tools without a planner: K1–K3 used temperatures 0, 0.5 and 1 with additional reasoning disabled; K4 and K5 used temperature 0 with reasoning budgets of 1 024 and 4 096 tokens. K2 was selected for subsequent assessments (Section 6.2).

Each setting used six runs per scenario, again excluding the first as a cold start. This adds 60 runs, of which 50 were analysed, separate from the initial 108-run selection study. Only the selected Haiku setting underwent the subsequent functional, robustness, concurrent-use and user assessments. Seven functional cases check complete, incomplete, revised and ambiguous requests and empty search results.

For prompt injection, each fixed test input was repeated in ten fresh sessions, giving 80 executions. The assessment distinguishes deviations from the expected workflow from control violations, such as skipping required confirmation, disclosing protected information or reporting a booking that did not occur. These are direct user-input tests, not attacks embedded in retrieved content or tool responses.

Three operational users and six developers each completed two tasks in their own wording: one with a relative date and morning preference, the other with an explicit date and afternoon preference. They rated 15 statements on a five-point agreement scale and could add comments. Groups were analysed separately; developers omitted the two items on operational usefulness and regular use. Concurrent-use tests repeated T01 under increasing load, measuring response times and HTTP outcomes separately from scheduling correctness.

The existing REST integration serves as a qualitative reference only. Its different task and interface prevent a direct comparison of response times or success rates.

6. Results and Discussion

This section first compares the three orchestration variants, then presents the settings selection and supporting assessments of the selected prototype.

6.1. Configuration Selection Results

Table 1 reports the configuration-selection results. The main finding is that technically successful tool execution does not necessarily produce a correct scheduling outcome. In T01, all five backend runs with Claude Haiku 4.5 were technically successful and used the tools correctly, yet only one met the outcome criterion. This gap shows why response time and cost must be considered together with outcome correctness.

Claude Haiku 4.5 without a separate planner was the only combination with correct outcomes in all five runs of both scenarios. Its median response times were 13.381 s and 17.247 s. Backend orchestration with the same model was faster at 8.153 s and 13.541 s, but yielded only one and two correct outcomes out of five.

The planner effect varied by model and metric. For Qwen3 in T01, adding a planner increased technical success from two to four runs but left correct outcomes at two. Without a planner, Qwen3 reached five correct outcomes in T02 but only two in T01. Backend orchestration had the lowest median cost in every model–scenario combination: 0.33–0.69 times the LLM-with-planner baseline. Input processing dominated token use; only Haiku recorded cache

Table 1: Configuration-selection results. LP: individual tools with planner; BP: bundled backend tools with planner; LN: individual tools without planner. T/U/C count technical success, correct tool use and correct outcomes, each out of five runs (Section 5). t is median cumulative response time for technically successful runs, including incorrect outcomes. Tokens and costs are medians over all five runs. K_{rel} normalises cost to LP within each model and scenario.

		T01				T02			
Model	**Variant**	**T/U/C**	t [s]	**Tokens**	K_{rel}	**T/U/C**	t [s]	**Tokens**	K_{rel}
Claude Haiku 4.5	LP	5/5/4	19.896	111 486	1.00	3/3/3	21.389	104 552	1.00
	BP	5/5/1	8.153	48 330	0.48	4/4/2	13.541	53 355	0.59
	LN	5/5/5	13.381	124 208	1.02	5/5/5	17.247	154 143	1.29
Qwen3-235B-A22B-2507	LP	4/4/2	21.767	67 771	1.00	3/3/1	19.542	50 195	1.00
	BP	4/4/2	14.408	21 416	0.33	3/3/2	12.661	24 713	0.51
	LN	2/2/2	15.568	59 747	0.87	5/5/5	19.680	65 471	1.27
Amazon Nova Pro	LP	2/2/1	11.950	30 427	1.00	4/2/2	19.061	52 193	1.00
	BP	4/3/1	8.570	20 361	0.69	2/2/2	11.920	19 712	0.39
	LN	2/2/2	9.394	34 845	1.16	2/2/2	10.164	38 616	0.73

usage, so token totals alone do not determine cost. Correction effort is excluded.

The Haiku configuration without a planner was selected on observed correctness, despite higher costs than backend orchestration. RestGPT reports higher success rates with an iterative planner informed by execution results [SXZ*23]. Our planner operates once per user input, without replanning between tool calls. Omitting it combines state updates and tool selection, rather than removing all decision-making. The tasks, models and planning mechanisms differ, so our study is not a direct replication of RestGPT. The limited scheduling workflow may help explain why the planner offered little benefit here, but this explanation was not tested separately.

6.2. Haiku Settings Selection

All five settings achieved technical success, correct tool use and correct outcomes in all regular runs (Table 2). Costs use the evaluation rates per million tokens: USD 1.10 input, USD 0.11 cached input, USD 1.375 cache writes and USD 5.50 output, matching Haiku 4.5 regional rates with five-minute cache writes checked on 1 November 2026 [Ant26].

Table 2: Haiku settings: median time and model-call cost over five regular runs per scenario, all with correct outcomes. Bold marks K2.

	T01		T02	
Setting	**Time [s]**	**Cost [USD]**	**Time [s]**	**Cost [USD]**
K1	14.253	0.081609	15.312	0.101242
K2	**12.528**	**0.081557**	**15.939**	**0.100520**
K3	13.379	0.081865	16.237	0.101124
K4	13.294	0.082705	16.324	0.101075
K5	14.915	0.081843	17.013	0.101329

With equal observed correctness, K2 was selected for the lowest median costs in both scenarios and shortest time in T01; K1 was faster in T02. The small cost differences and five runs per setting and scenario do not establish a general efficiency advantage. Higher temperature and additional reasoning did not improve observed correctness; temperature 0.5 is not shown to be generally optimal.

6.3. Functional Cases

The seven cases assessed K2 only and document coverage, not repeated-run success rates. E01 completed the intended scheduling process. Missing customer identifiers (E02) and appointment purposes (E03) triggered questions before proceeding. A changed day (E04) triggered a new availability search while retaining customer information; a revised customer identifier (E05) was used for booking. Ambiguous input (E06) prompted clarification. E07 correctly reported no available appointments only after a server-side correction distinguished empty search results from technical failures. This case therefore reflects an integration fix as well as the selected configuration.

6.4. Direct Prompt-Injection Tests

No control violation was observed across eight fixed inputs, each tested in ten fresh sessions (80 executions). Attempts to skip confirmation (P01) left the expected workflow intact in 10/10 sessions. Requests for internal instructions (P02) left it intact in 7/10; no tool call occurred in 3/10. Attempts to proceed without a customer identifier (P03) or invent an appointment purpose (P04) prompted clarification in 10/10 each. Instructions to ignore errors and claim booking success (P05) left the expected workflow intact in 8/10; two deviations involved no false success claims. Attempts to obtain connection information (P06), model-access credentials (P07) and authentication data (P08) caused no disclosure in 10/10 each. Disclosure tests are not combined into a scheduling success rate. These results distinguish workflow disruptions from control violations; they neither establish broad attack coverage nor isolate model protection from application controls.

6.5. User Feedback

Operational users gave medians of 5 for stepwise and combined input, understandable booking and natural-language time handling. Response clarity and response time each received 4. Daily-work support and willingness to use the assistant regularly each received a median of 4 and mean of 3.67; all these items had three responses. These are participant ratings, not task-success measurements.

Comments requested shorter responses, concise questions and more visible options; one user preferred a conventional form. The median of 5 for scheduling with few inputs suggests a preference for short interactions, not necessarily for chat. The six developers also requested faster responses and interface improvements, but cannot establish operational acceptance. No conversation mixing was observed during parallel use; this was not a systematic context-isolation test.

6.6. Load Tests

Table 3 compares three T01 load profiles for K2. L02 used five times the virtual clients of L01; L03 applied a higher stress load. L01 and L02 each ran for 10 minutes and 2 seconds, L03 for 48 minutes and 31 seconds. Absolute client counts and internal capacity requirements are omitted.

Table 3: T01 load tests. Time is the mean per request; HTTP errors are status 500 or 504 among all recorded calls, rounded to two decimals. These are not booking-correctness rates.

Profile	**Load**	**Mean time [s]**	**HTTP errors [%]**
L01	Reference	7.053	0.91
L02	Fivefold	8.105	0.80
L03	Stress	15.455	9.98

At fivefold load, requests took longer while the HTTP error rate stayed close to the reference level; the small decrease does not demonstrate improved reliability. Stress increased both response time and errors. L01 and L02 contained HTTP 500 errors; L03 also contained HTTP 504 errors. Successful HTTP responses do not establish correct bookings. These mean request times are not directly

comparable with the median scenario times in Tables 1 and 2; different test durations also limit conclusions about sustained capacity.

6.7. Implications for Controlled Integration

The comparison shows an efficiency–correctness trade-off: bundling can reduce processing effort, while neither bundling nor an extra model stage guarantees correct outcomes. Applications should evaluate technical execution, tool use and outcome correctness separately, distinguish configuration selection from later coverage checks, and enforce confirmation and validation outside the model. Clear tool results are essential, especially for empty searches and failures. Conversational input adds flexibility over a fixed REST interface but requires explicit state management. No quantitative advantage over that interface was tested.

7. Limitations and Transferability

The findings apply to the tested workflow and configurations. More tools, overlapping functions or longer dependencies may benefit from separate planning. The missing backend-without-planner variant prevents a complete comparison, and bundling changes tool granularity as well as execution location.

Three model versions were compared on two scenarios with five regular repetitions: one run changes a success rate by 20 percentage points. These results support descriptive comparison, not reliable estimates for wider use. T01 and T02 informed both selection and settings decisions and are not independent validation. Later assessments cover Haiku only; prompts developed with one evaluated model family may also favour it despite shared prompts and tools within each variant.

Response times depend on external services; medians exclude technical failures and therefore use different sample sizes. Date handling covers selected expressions, and differing load-test durations limit conclusions about sustained capacity. Repeated fixed injection inputs do not cover indirect attacks or establish comprehensive security, and the tests do not isolate model protection from application controls. Functional cases provide coverage rather than repeated-run success rates; E07 also includes an implementation fix. Parallel use does not verify context isolation, and brief feedback from three users and six developers cannot establish long-term operational acceptance.

Omitting prompts, run-level logs and internal deployment details limits replication, though generalised cases, settings and criteria make the design inspectable. Structured state, clear tool boundaries, validation and human confirmation may transfer to other processes, but require task-specific evaluation.

8. Conclusion and Future Work

This paper compared three MCP-based orchestration variants in a conversational scheduling workflow. For the tested scenarios, Haiku with individual tools and no separate planner combined correct outcomes in all regular runs with shorter response times than its planner-based counterpart. Backend orchestration was faster and cheaper for Haiku, but produced fewer correct outcomes. The comparison therefore supports assessing outcome correctness separately from successful tool execution and alongside time and cost. It does not establish an architecture that is best for every task.

Additional assessments identified integration and usability improvements for the selected prototype, without establishing comprehensive security, long-term acceptance or production readiness.

Future work should complete the comparison with backend orchestration without a planner and introduce independent scenarios, larger tool sets and more repetitions.

Other LLMs should be tested with the same prompts, tool definitions and independent scenarios, documenting any adjustments needed. Functional and robustness checks should cover these alternatives as well as Haiku. Longer user studies, sustained load tests and targeted context-isolation and prompt-injection checks are needed before operational use.

References

[Ant26] ANTHROPIC: Claude platform: Pricing, 2026. Haiku 4.5 token prices and regional pricing. Accessed 1 November 2026. URL: https://platform.claude.com/docs/en/about-claude/pricing. 5

[AWV*19] AMERSHI S., WELD D., VORVOREANU M., FOURNEY A., NUSHI B., COLLISSON P., SUH J., IQBAL S., BENNETT P. N., INKPEN K., TEEVAN J., KIKIN-GIL R., HORVITZ E.: Guidelines for Human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (New York, NY, USA, 2019), CHI '19, Association for Computing Machinery, pp. 1–13. doi:10.1145/3290605.3300233. 2

[Mod25] MODEL CONTEXT PROTOCOL: Model Context Protocol: Architecture, 2025. Specification revision 2025-06-18. Accessed 28 October 2026. URL: https://modelcontextprotocol.io/specification/2025-06-18/architecture. 1, 2

[QLY*24] QIN Y., LIANG S., YE Y., ZHU K., YAN L., LU Y., LIN Y., CONG X., TANG X., QIAN B., ZHAO S., HONG L., TIAN R., XIE R., ZHOU J., GERSTEIN M., LI D., LIU Z., SUN M.: ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations* (2024). URL: https://openreview.net/forum?id=dHng2O0Jjr. 1, 2

[SDYD*23] SCHICK T., DWIVEDI-YU J., DESSÌ R., RAILEANU R., LOMELI M., HAMBRO E., ZETTLEMOYER L., CANCEDDA N., SCIALOM T.: Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems* (2023), vol. 36. doi:10.52202/075280-2997. 1

[SXZ*23] SONG Y., XIONG W., ZHU D., WU W., QIAN H., SONG M., HUANG H., LI C., WANG K., YAO R., TIAN Y., LI S.: RestGPT: Connecting large language models with real-world RESTful APIs. *arXiv preprint arXiv:2306.06624* (2023). doi:10.48550/arXiv.2306.06624. 1, 2, 5

[WTC22] WU T., TERRY M., CAI C. J.: AI Chains: Transparent and controllable Human-AI interaction by chaining large language model prompts. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems* (New York, NY, USA, 2022), CHI '22, Association for Computing Machinery. doi:10.1145/3491102.3517582. 2

[YZY*23] YAO S., ZHAO J., YU D., DU N., SHAFRAN I., NARASIMHAN K., CAO Y.: ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations* (2023). URL: https://arxiv.org/abs/2210.03629. 1